

AI IN ACTION - DAY 12

Instruction dành cho học viên

Cloud Helpdesk Agent: từ localhost đến deployment-ready service

Bài tập lý thuyết 2 giờ - nhẹ, mở, free-first, có code/khuôn mẫu sẵn. Mục tiêu là hiểu quy trình triển khai một AI agent system thật, không biến buổi lý thuyết thành lab dài.

Free-first

Không bắt buộc tài khoản trả phí

Docker-first

Local Docker là chuẩn pass tối thiểu

Open-ended

Tự chọn stack theo bài toán & chi phí

1. Mục tiêu và bối cảnh bài tập

Trong slide Day 12, câu hỏi mở đầu là: agent chạy trên laptop rồi thì khi nào 100 người dùng được? Bài tập này giúp bạn trả lời câu hỏi đó bằng một service mẫu có thể chạy local bằng Docker, kiểm tra health/readiness, dùng API key, cấu hình bằng env vars, log có cấu trúc và có phương án deploy hoặc giả lập deploy.

Kết quả quan trọng nhất

Bạn không cần chứng minh rằng mình đã dùng được một cloud provider cụ thể. Bạn cần chứng minh rằng mình hiểu agent từ localhost muốn đi đến production phải có container, config/secrets đúng cách, health check, auth/rate limit/cost guard, log, và một lựa chọn deployment stack hợp lý.

Phần trong slide	Bạn sẽ thực hành bằng gì trong bài này
Cloud infrastructure	Nhìn service theo các lớp: compute, storage, networking, security/IAM, observability.
Localhost -> production	Chuyển từ chạy bằng laptop sang service chạy trong container, có config, health, auth, logs.
Docker & containerization	Build image, chạy Docker Compose, dùng .dockerignore, không đưa .env vào image/repo.
API gateway & security	Dùng X-API-Key, nhận biết 401, đặt rate limit/cost guard ở mức demo.
Scaling & reliability	Phân biệt /health và /ready, hiểu stateless, replicas, graceful shutdown.
Cloud deployment options	Chọn một stack phù hợp với use case, chi phí, kỹ năng team và yêu cầu vận hành.

Thời lượng gợi ý: 2 giờ sau phần nghe giảng lý thuyết. Bài thực hành chuyên sâu 4 giờ sẽ đi sâu hơn; ở đây ưu tiên đọc hiểu, chạy mẫu, quan sát hành vi production và tự đưa ra quyết định thiết kế.

2. Chính sách free-first

Bài tập được thiết kế để hoàn thành được bằng tài khoản miễn phí hoặc mô phỏng open-source. Public cloud URL là phần khuyến khích, không phải điều kiện bắt buộc để pass phần lý thuyết.

Path	Hạ tầng	Bạn cần nộp gì	Khi nào chọn
A - Zero Cloud	Docker/Docker Compose local	Screenshot/curl log: /health, /ready, /ask, 401 khi thiếu key, check_production_ready.py	Tất cả học viên; không cần cloud account
B - Free PaaS	Render hoặc Railway free/free trial	Public URL hoặc deploy log + env config + health path	Khi tài khoản, quota và xác minh cho phép
C - OSS DevOps Simulation	act/Gitea Actions, Grafana LGTM, Ollama hoặc Mock LLM	Local CI/observability/demo flow, không cần public URL	Muốn hiểu DevOps production-like nhưng không phụ thuộc cloud

Lưu ý về cloud miễn phí

Chính sách free tier và trial có thể thay đổi. Nếu cloud provider yêu cầu billing, hết credit, quota hoặc xác minh tài khoản, hãy dùng Path A hoặc Path C và giải thích quyết định của bạn trong báo cáo nộp bài.

3. Ví dụ chính: Cloud Helpdesk Agent

Bạn là AI Engineer trong một công ty nhỏ. Team muốn triển khai một agent hỏi đáp nội bộ giúp nhân viên hỏi về quy trình IT, policy công ty, lỗi thường gặp và hướng dẫn sử dụng công cụ. Trong lớp, agent dùng Mock LLM để không phát sinh chi phí model; nhưng kiến trúc vẫn mô phỏng cách deploy một AI agent system thật.

Thành phần	Trong starter kit	Ý nghĩa production
API service	FastAPI app với /health, /ready, /ask	Service có interface rõ ràng để LB/API gateway kiểm tra.
Auth	Header X-API-Key	Không để endpoint AI mở công khai không kiểm soát.
Config/secrets	APP_API_KEY đọc từ env hoặc secret file	Không hardcode key trong code, compose hoặc repo.
Logging	Structured logs có request_id	Dễ debug request, latency, lỗi và chi phí.
Cost guard	Giới hạn input và mock budget	Tránh request dài hoặc spam làm tăng token/cost.
Container	Dockerfile, docker-compose.yml, .dockerignore	Build once, run anywhere, giảm lệch dev/prod.
Readiness check	/ready kiểm tra app và dependency	Không route traffic vào service chưa sẵn sàng.

Lệnh chạy nhanh

```
# 1) Unzip starter kit
unzip cloud_ai_agent_starter_kit_secure.zip
cd cloud_ai_agent_starter_kit_secure

# 2) Create local secret - do not commit .env
cp .env.example .env
# Edit .env and set APP_API_KEY to a long demo value.

# 3) Run local service
docker compose up --build

# 4) Check main endpoints
curl http://localhost:8000/health
curl http://localhost:8000/ready

# 5) Test auth fail/success
curl -X POST http://localhost:8000/ask -H "Content-Type: application/json" -d '{"question": "Hello"}'

curl -X POST http://localhost:8000/ask -H "Content-Type: application/json" -H "X-API-Key: <APP_API_KEY from .env>" -d '{"question": "When should we choose Cloud Run over Render?"}'
```

Không leak secret

Không chụp màn hình hoặc nộp file có key thật. Không đưa .env lên Git. Không để APP_API_KEY literal trong docker-compose.yml. docker compose config có thể in ra env đã resolve, nên tránh nộp output có secret thật.

4. Timeline bài tập 2 giờ

Giảng viên có thể điều chỉnh tốc độ, nhưng bạn nên giữ mục tiêu: chạy được tối thiểu Path A, sau đó dành thời gian để đọc, chọn stack và giải thích trade-off.

Mốc thời gian	Việc cần làm	Output nhỏ
0-10 phút	Đọc scenario, mở starter kit, kiểm tra Docker, tạo .env từ .env.example.	Hiểu file nào chứa app, Dockerfile, compose, config.
10-30 phút	Chạy docker compose up --build; gọi /health, /ready, /ask.	Có curl output thành công và 401 khi thiếu key.
30-45 phút	Đọc Dockerfile, docker-compose.yml, .dockerignore, settings.py.	Ghi lại 3 điều đã làm đúng và 1 rủi ro nếu đưa lên production.
45-75 phút	Làm Stack Decision Canvas: chọn Render/Railway/Cloud Run/ECS/K8s/local simulation tùy bài toán.	Một bảng quyết định stack + lý do + cost drivers.
75-100 phút	Chọn Path B hoặc C nếu muốn: deploy PaaS, hoặc mô phỏng CI/observability local.	Public URL hoặc bằng chứng mô phỏng production-like.
100-115 phút	Chạy check_production_ready.py, rà soát secret, log, health, auth.	Checklist pass cơ bản.
115-120 phút	Chuẩn bị demo 90 giây và nộp báo cáo ngắn.	README/screenshot/curl + stack decision.

Báo cáo nộp bài nên ngắn, không quá 1-2 trang hoặc một README rõ ràng. Bạn có thể nộp repo link, screenshot, curl output, hoặc video ngắn tùy lớp yêu cầu.

5. Yêu cầu bắt buộc tối thiểu

- Service chạy được local bằng Docker Compose.
- GET /health trả về trạng thái service và không phụ thuộc vào secret thật.
- GET /ready cho biết service có sẵn sàng nhận traffic hay không.
- POST /ask yêu cầu header X-API-Key; thiếu/sai key phải trả 401/403.
- APP_API_KEY không hardcode trong source code hoặc docker-compose.yml; dùng .env local hoặc env var từ platform.
- Có structured logs hoặc log đủ để thấy request_id, endpoint, status, latency cơ bản.
- Chạy script production readiness hoặc tự đối chiếu checklist tương đương.
- Hoàn thành Stack Decision Canvas ở mức lightweight.

6. Stack Decision Canvas - phần nền bổ sung

Vì sao cần phần này?

Slide đã có mục 'Cloud deployment options' và mục tiêu so sánh Railway, Render, AWS ECS, Serverless. Tuy nhiên khi đi làm thật, câu hỏi không chỉ là 'deploy được ở đâu?' mà là 'stack nào phù hợp với hệ thống, team và ngân sách công ty?'. Vì vậy trong bài tập này, mỗi nhóm cần làm một decision canvas ngắn trước khi chọn stack.

Cách làm: chọn một context giả định, ví dụ công ty có 100 nhân viên nội bộ, 500 request/ngày, cần uptime giờ hành chính, dữ liệu không quá nhạy cảm, budget ban đầu thấp, team 1-2 người. Sau đó so sánh ít nhất 2 deployment stacks.

Câu hỏi	Gợi ý trả lời ngắn
1. Người dùng và traffic?	Bao nhiêu user, request/ngày, giờ cao điểm, latency kỳ vọng.
2. Dữ liệu có nhạy cảm không?	Có PII, dữ liệu khách hàng, nội bộ, yêu cầu region/compliance không.
3. Agent dùng tài nguyên gì?	LLM API, local model, vector DB, Redis, database, file storage, background worker.
4. Team vận hành mạnh đến đâu?	Có DevOps/SRE không, có quen AWS/GCP/Kubernetes không, cần đơn giản hay kiểm soát sâu.
5. Chi phí chính đến từ đâu?	LLM tokens, compute uptime, DB/vector DB, egress, logs/traces, CI/CD, support/SLA.
6. Rủi ro triển khai lớn nhất?	Cold start, secret leak, hết free tier, lock-in, thiếu monitoring, không rollback được.

Bảng chấm nhanh để tự chọn stack

Tiêu chí	Trọng số	Stack A	Stack B	Stack C
Thời gian deploy MVP	15%	__ / 5	__ / 5	__ / 5
Dễ vận hành với team hiện tại	20%	__ / 5	__ / 5	__ / 5
Dự đoán và kiểm soát chi phí	20%	__ / 5	__ / 5	__ / 5
Scale/reliability khi tăng user	20%	__ / 5	__ / 5	__ / 5
Security/compliance/secrets	15%	__ / 5	__ / 5	__ / 5
Khả năng migrate sau này	10%	__ / 5	__ / 5	__ / 5
Kết luận	100%	Chọn/Không	Chọn/Không	Chọn/Không

7. Gợi ý chọn deployment stack

Bạn được phép chọn phương án khác nếu giải thích được. Bảng dưới đây là định hướng cho người mới bắt đầu, không phải đáp án duy nhất.

Stack	Phù hợp khi	Điểm mạnh	Cần lưu ý
Docker Compose local	Lớp học, prototype, demo không cần public URL	Miễn phí, kiểm soát dễ, học rõ container/env/logs	Không phải production thật, không có public traffic/managed scaling
Render/Railway	MVP, side project, demo cho stakeholder, team nhỏ	Deploy nhanh từ GitHub, env vars/health check dễ cấu hình	Free tier/trial có quota, spin down hoặc credit; không mặc định phù hợp production quan trọng
Cloud Run	HTTP container stateless, traffic biến động, muốn scale to zero	Serverless containers, trả theo usage, phù hợp service stateless	Cần hiểu billing, cold start, concurrency, region và IAM
ECS/Fargate	Công ty dùng AWS, cần kiểm soát mạng/IAM sâu hơn	Container managed, phù hợp production có VPC, ALB, CloudWatch	Thiết lập phức tạp hơn PaaS, cần estimate vCPU/RAM/storage/logs
Kubernetes/EKS/GKE/AKS	Nhiều service, traffic lớn, platform team mạnh	Orchestration mạnh, rollout/rollback/replicas/ingress chuẩn	Overkill cho MVP; chi phí vận hành và độ phức tạp cao

Rule of thumb

- MVP/learning: chọn Docker Compose local trước, sau đó Render/Railway nếu cần public URL.
- Internal tool ít traffic nhưng cần scale-to-zero: cân nhắc Cloud Run hoặc serverless container tương đương.
- Production trong công ty đã dùng AWS/GCP/Azure: ưu tiên stack đang có IAM, network, logging, billing và vận hành sẵn.
- Không chọn Kubernetes chỉ vì 'ngầu'. Chọn khi thật sự cần orchestration và có người vận hành.
- Luôn estimate cost trước: compute, database/vector DB, LLM tokens, logs/traces, egress, storage, monitoring, support.

8. Cost thinking cho AI agent

Với AI system, chi phí không chỉ là server. LLM/API tokens, embedding, vector DB, logging/tracing và request bất thường có thể làm chi phí tăng nhanh. Bài tập yêu cầu bạn liệt kê cost drivers thay vì tính hóa đơn chính xác tuyệt đối.

Cost driver	Câu hỏi cần hỏi	Guardrail gợi ý
LLM tokens	Mỗi request dùng bao nhiêu input/output tokens? Có retry không?	Giới hạn độ dài input, budget per user, cache câu hỏi thường gặp.
Compute uptime	Service chạy 24/7 hay scale to zero? Cần min instances không?	Tắt service demo sau lớp, scale to zero nếu phù hợp.
Database/vector DB	Dữ liệu lớn hay nhỏ? Cần managed DB không?	Bắt đầu SQLite/local/mock; production mới dùng managed DB/vector DB.
Logs/traces	Log bao nhiêu request? Có lưu PII không?	Log sampling, retention ngắn cho dev, scrub PII.
Egress/bandwidth	Có gửi file/video/image ra ngoài không?	CDN/cache, giới hạn file size, region gần user.
Support/SLA	Ứng dụng có quan trọng không? Ai trực khi down?	MVP không cần enterprise SLA; production cần alert/runbook/on-call.

Mini worksheet ước lượng chi phí

Input	Giá trị nhóm tự điền	Ghi chú
Số user / ngày	_____	Ví dụ: 100 nhân viên nội bộ
Request / user / ngày	_____	Ước lượng giờ cao điểm
Avg input/output tokens	_____	Dùng để tính LLM cost nếu không dùng Mock LLM
Compute mode	24/7 / scale-to-zero / scheduled	Ảnh hưởng lớn đến chi phí
DB/vector/logs	local / managed / external SaaS	Tách rõ dev và prod
Budget guard	rate limit / max prompt / quota	Phải có trước khi public

9. Mốc điểm cộng

Điểm cộng tối đa: 15. Giảng viên có thể áp dụng theo cách 'đạt mức cao nhất thì lấy mức đó', không cộng dồn 5+10+15. Bạn chỉ cần chọn mục tiêu phù hợp với thời gian và trình độ nhóm.

Mốc	Mục tiêu	Demo/Minh chứng cần có	Cách tự kiểm
+5	Production hygiene & basic security	Không hardcode secrets; .env không commit; /health và /ready hoạt động; X-API-Key đúng/sai; Dockerfile có multi-stage hoặc non-root; logs có request_id.	Chạy local, gửi 3 curl: health, ask thiếu key, ask đúng key; chụp screenshot không lộ secret.
+10	Cloud-ready workflow & observability	Đạt +5, thêm public URL trên Render/Railway/Cloud Run hoặc mô phỏng CI/observability local; có README deploy; có rate limit/cost guard demo; log có latency/status.	Demo public URL hoặc local CI/observability; giải thích env vars và health path ở platform.
+15	Production pattern extension	Đạt +10, thêm một pattern rõ: async worker + queue, 2 replicas + load balancer, rollback/canary, OpenTelemetry dashboard, IaC, hoặc stack cost comparison sâu.	Demo pattern hoạt động hoặc trình bày trade-off + rủi ro + cách rollback/cost control.

Không bắt buộc deploy cloud để được điểm cộng

Nếu cloud free tier bị giới hạn, bạn vẫn có thể đạt +10 hoặc +15 bằng mô phỏng production-like local: CI chạy bằng act/Gitea Actions, observability bằng Grafana LGTM, local LLM/Ollama hoặc Mock LLM, Docker Compose nhiều service.

10. Checklist trước khi nộp

Nhóm	Checklist	Minh chứng
Run	docker compose up --build chạy được; service không crash khi gọi /health	Terminal screenshot hoặc log
Security	APP_API_KEY lấy từ env/.env; không hardcoded; .env nằm trong .gitignore/.dockerignore	File config hoặc screenshot đã che key
API	/health, /ready, /ask hoạt động; thiếu/sai key trả lỗi	curl output
Observability	Log có request_id/status/latency hoặc thông tin tương đương	Log sample
Deployment	Path A/B/C rõ ràng; public URL nếu có	URL/log/screenshot
Stack decision	Có Canvas: requirement, 2-3 options, cost drivers, rủi ro, lựa chọn cuối	Bảng trong README/report
Reflection	Nêu 2 điều học được và 1 điều sẽ cải thiện nếu đưa production thật	Đoạn văn ngắn

Mẫu README/report nộp bài

Cloud Helpdesk Agent - Student Submission

1. Selected path

- Path A / Path B / Path C:
- Why this path:

2. How to run

- docker compose up --build
- curl /health:
- curl /ready:
- curl /ask with X-API-Key:

3. Stack Decision Canvas

- Requirements:
- Options compared:
- Cost drivers:
- Risks:
- Final decision:

4. Bonus milestone, if any

- +5 / +10 / +15:
- Demo evidence:

5. Reflection

- 2 things learned:
- 1 thing to improve for real production:

11. Troubleshooting nhanh

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
Service báo thiếu APP_API_KEY	Chưa copy .env.example thành .env hoặc chưa set env	cp .env.example .env, sửa APP_API_KEY, restart docker compose
curl /ask trả 401	Thiếu header X-API-Key hoặc key sai	Gửi đúng header; không copy khoảng trắng thừa
Port bị chiếm	Máy đã có service dùng 8000	Đổi mapping trong compose hoặc tắt service cũ
Deploy cloud fail vì PORT	App hardcode port thay vì đọc env PORT	Đảm bảo app đọc os.getenv('PORT', 8000)
Public URL chậm lần đầu	Free instance spin down/cold start	Chờ spin up; ghi nhận đây là trade-off của free tier
Không tạo được cloud account	Provider yêu cầu xác minh/quota/billing	Dùng Path A hoặc C; không bị trừ nếu nộp minh chứng local/open-source

Demo 90 giây

- Mở terminal: show docker compose đang chạy hoặc public URL nếu có.
- Gọi /health và /ready để chứng minh service có thể được load balancer/platform kiểm tra.
- Gửi một request thiếu X-API-Key để thấy auth fail, rồi gửi request đúng key để thấy response.
- Chỉ ra nơi secret được cấu hình ngoài code và nói rõ stack bạn chọn trong Decision Canvas.
- Nêu 1 trade-off: chi phí, cold start, vận hành, observability hoặc khả năng scale.

12. Tài liệu tham khảo

Nguồn từ slide bài giảng

- [1] Day 12 slide deck: mục tiêu ngày học gồm cloud infrastructure, gap dev -> production, Dockerfile, cloud options, API gateway/security và deploy public URL.
- [2] Day 12 slide deck: deliverable cuối ngày yêu cầu containerized agent, health check, basic authentication, env vars đúng cách và public URL/demo.
- [3] Day 12 slide deck: 12-Factor App cho AI Agent nhấn mạnh config in env, stateless processes, port binding và dev/prod parity.
- [4] Day 12 slide deck: Lab 12 yêu cầu Dockerfile, local build/test, Railway/Render, /health, X-API-Key và demo request-response.
- [5] Day 12 slide deck: platform comparison có Railway, Render, Cloud Run, Lambda với khác biệt về pricing, scaling, complexity và best-fit.

Nguồn chính thức để tự cập nhật giá và thực hành

- [6] Docker Docs - Multi-stage builds: <https://docs.docker.com/build/building/multi-stage/>
- [7] Render Docs - Deploy for Free: <https://render.com/docs/free>
- [8] Render Docs - Web Services: <https://render.com/docs/web-services>
- [9] Railway Docs - Free Trial: <https://docs.railway.com/pricing/free-trial>
- [10] Railway Docs - Pricing Plans: <https://docs.railway.com/pricing/plans>
- [11] Google Cloud - Cloud Run Pricing: <https://cloud.google.com/run/pricing>
- [12] Google Cloud Pricing Calculator: <https://cloud.google.com/products/calculator>
- [13] AWS Fargate Pricing: <https://aws.amazon.com/fargate/pricing/>
- [14] Amazon ECS Pricing: <https://aws.amazon.com/ecs/pricing/>
- [15] AWS Pricing Calculator: <https://calculator.aws/>
- [16] AWS Well-Architected - Cost Optimization: <https://docs.aws.amazon.com/wellarchitected/latest/framework/cost-optimization.html>
- [17] Kubernetes Docs - Liveness, Readiness, and Startup Probes: <https://kubernetes.io/docs/concepts/workloads/pods/probes/>
- [18] Microsoft Learn - Azure pricing calculator: <https://learn.microsoft.com/en-us/azure/cost-management-billing/costs/pricing-calculator>

Nguyên tắc khi dùng pricing

Không copy giá một lần rồi xem là đúng mãi. Trước khi deploy production, luôn kiểm tra trang pricing chính thức, region, billing mode, free tier/trial, quota, support/SLA và chính sách sử dụng của công ty.